

AI Attack Lab

一、环境准备

本次实验大家无需在Linux环境下做，在自己的本机上完成即可。

首先大家需要在自己的电脑上准备pytorch环境。这一步大家根据自己的电脑类型自行搜索教程完成，需要注意的是本次实验只需要大家使用CPU训练和测试模型即可，因此安装CPU版本的pytorch即可。

可以通过运行实验文件中的`model.py`来测试环境是否配置成功，正常情况下，其输出如下：

```
(pytorch) C:\User...>python model.py
Using cpu device
Train Epoch: 1  Loss: 0.212456
Test Accuracy = 9669 / 10000 = 0.9669
Train Epoch: 2  Loss: 0.072187
Test Accuracy = 9852 / 10000 = 0.9852
Train Epoch: 3  Loss: 0.039394
Test Accuracy = 9896 / 10000 = 0.9896
Train Epoch: 4  Loss: 0.004776
Test Accuracy = 9926 / 10000 = 0.9926
Train Epoch: 5  Loss: 0.021490
Test Accuracy = 9927 / 10000 = 0.9927
```

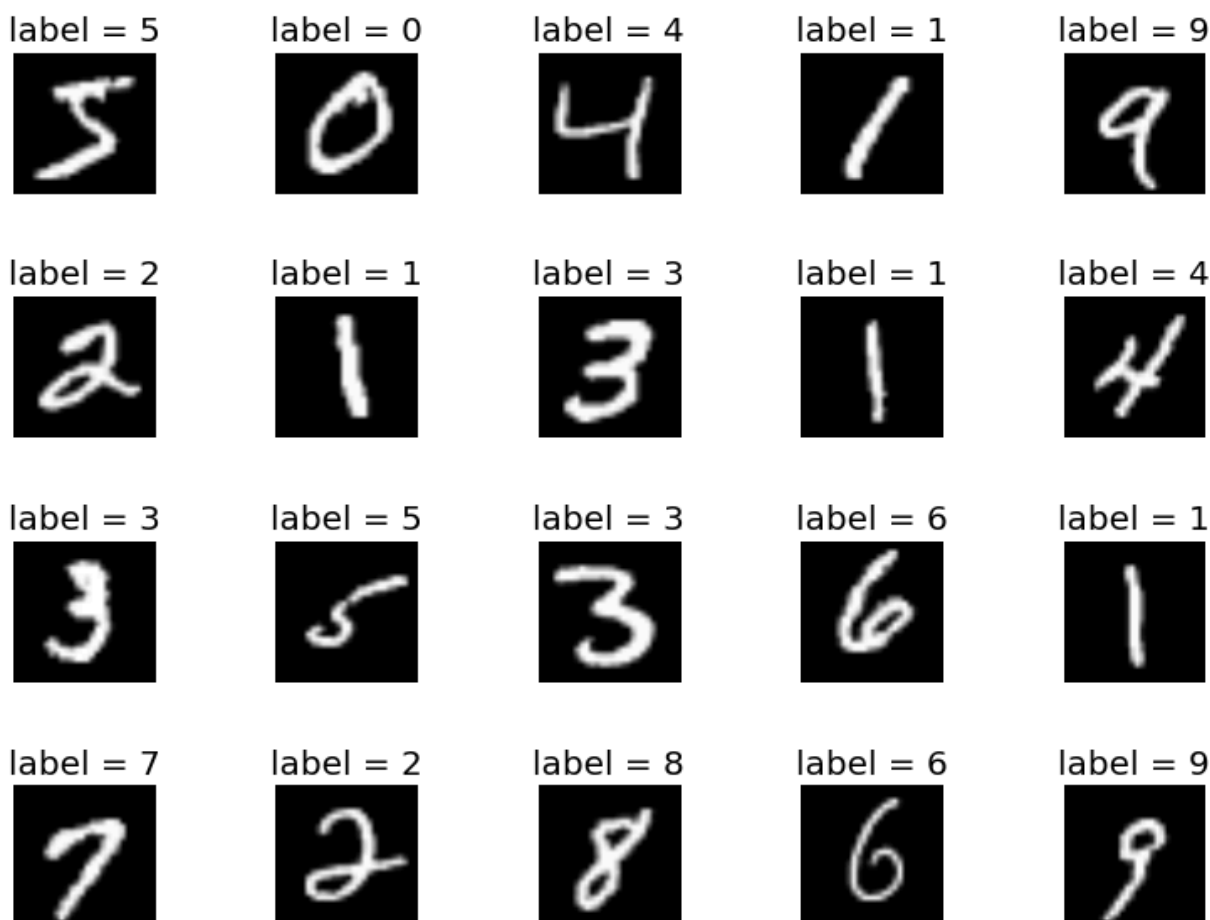
二、模型和数据集介绍

本次实验采用的模型见`model.py`，具体结构如下：

```
1 class Net(nn.Module):
2     def __init__(self):
3         super(Net, self).__init__()
4         self.conv1 = nn.Conv2d(1, 32, 3, 1)
5         self.conv2 = nn.Conv2d(32, 64, 3, 1)
6         self.dropout1 = nn.Dropout(0.25)
```

```
7         self.dropout2 = nn.Dropout(0.5)
8         self.fc1 = nn.Linear(9216, 128)
9         self.fc2 = nn.Linear(128, 10)
10
11     def forward(self, x):
12         x = self.conv1(x)
13         x = F.relu(x)
14         x = self.conv2(x)
15         x = F.relu(x)
16         x = F.max_pool2d(x, 2)
17         x = self.dropout1(x)
18         x = torch.flatten(x, 1)
19         x = self.fc1(x)
20         x = F.relu(x)
21         x = self.dropout2(x)
22         x = self.fc2(x)
23         output = F.log_softmax(x, dim=1)
24         return output
```

使用的数据集为MNIST，该数据集是大型手写数字数据库，其包含0~9十个数字，具体文件见 **data** 文件夹：



https://blog.csdn.net/weixin_44862382

`model.py` 利用MNIST数据集对上述模型进行训练和测试，结果如第一张图所示。

三、task 1 对抗样本攻击

首先我们需要实现对抗样本攻击中的**Fast Gradient Sign Attack (FGSM)**。FGSM的思想是在输入图片增加一些扰动，使得模型对于图片的预测发生错误。

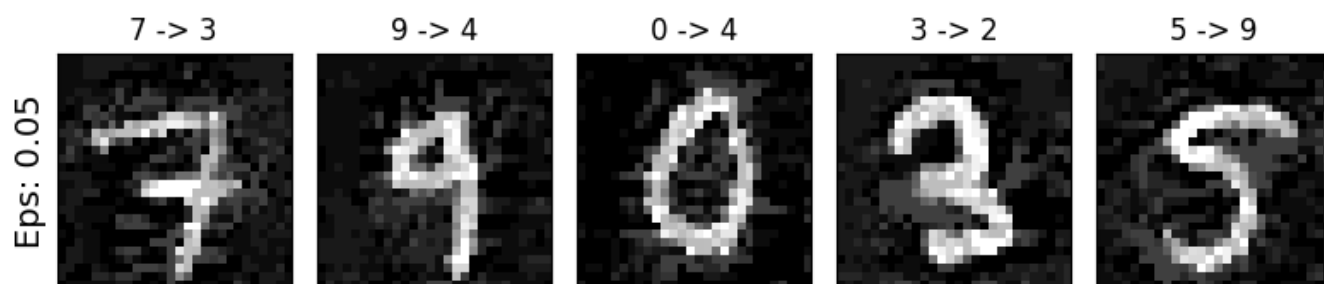
$$x' = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

其中 x 是原始输入图像； y 是图像 x 的真实标签； θ 是神经网络的参数； $J(\theta, x, y)$ 是用于训练神经网络的损失函数，衡量模型预测结果与真实标签之间的差距。攻击方式是先计算出损失函数对输入 x 的梯度 $\nabla_x J(\theta, x, y)$ ，然后在和梯度符号相同的方向上增加一个扰动，步长为 ϵ 。使得模型对 x' 的预测结果不等于 y 。

3.1 untargeted attack

实现untargeted Fast Gradient Sign Attack，即只需要让模型对图片的预测发生错误即可，不需要关注预测的结果。例如对于一张标签为1的图片，你只需要在其上面增加扰动，让模型预测结果不为1，至于结果究竟是哪个值不用关心。

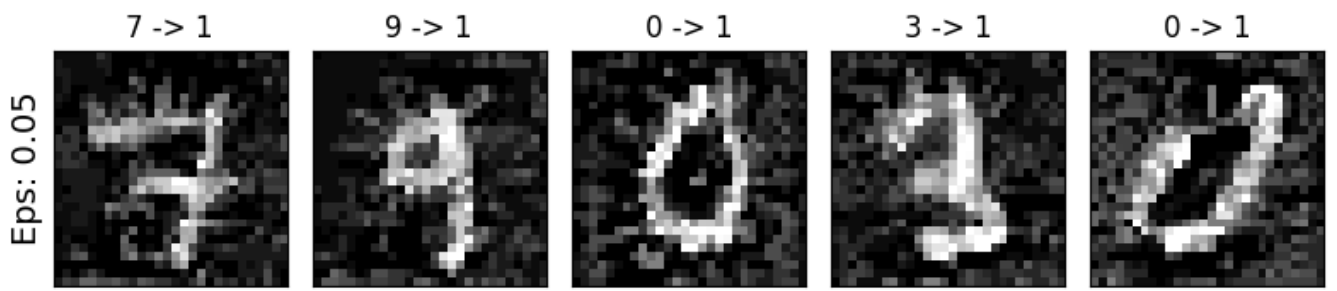
调整你的 ϵ 的取值，要求在 ϵ 尽可能小时让错误率尽可能大。基本要求：在 $\epsilon = 0.05$ 时错误率超过95%。同时检查增加扰动后的图片，要求不影响人眼辨认。



3.2 targeted attack

实现targeted Fast Gradient Sign Attack，即需要让模型对图片的预测结果为一个固定值。例如对于所有预测结果不是1的图片，无论其标签是什么，都通过在上面加扰动的方式让模型的预测结果变成1。此时对输入增加扰动的方式应该有所变化。

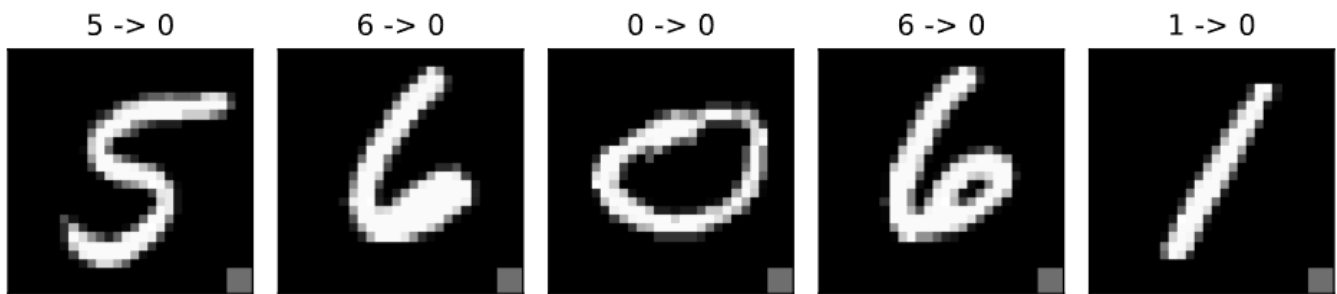
调整你的 ϵ 的取值，要求在 ϵ 尽可能小时让成功率（预测结果为你设定的固定值的图片数/总图片数）尽可能大。基本要求：在 $\epsilon = 0.05$ 时成功率超过95%。同时检查增加扰动后的图片，要求不影响人眼辨认。



四、task 2 后门攻击

现在我们实现后门攻击。所谓后门攻击就是在模型训练时，在一部分训练数据上（另次占比为 η ）增加一个特定的trigger。在训练时，对于这些带有trigger的输入，让模型将它们映射到某一个具体结果。例如，在一部分的图片的右下角加一个小方框作为trigger，对于这些有trigger的图片，训练时让它们都映射到1。这样当使用模型进行预测时，只要往图片右下角加入相同的小方框，无论该图片的标签实际是多少，都能让模型对该图片的预测结果为1。

调整 η 的值，要求在 η 尽可能小时成功率（预测结果为你设定的固定值的图片数/总图片数）尽可能大。基本要求，在 $\eta = 0.01$ 时成功率超过95%。你可以自己选择要加入的trigger，下面一种示例。



可以看到在右下角加了一个小方框，但是这个trigger其实比较突兀，要求大家选择看起来更为自然的trigger。